

複雑システム系演習 1

第4回 確率的 ARMS

鈴木 泰博

☆ 今回やること

決定的 ARMS と確率的 ARMS の違いを理解する
反応速度論にもとづいて反応規則を確率的に選ぶ仕組みを実装する
ルーレット選択（イカサマさいころ）で確率的 ARMS を完成させる

決定的 ARMS と確率的 ARMS の違い

第3回で作った決定的 ARMS は「適用できる反応規則をすべて毎回適用する」ものでした。しかし現実の化学反応では、反応は確率的に起きます。

なぜ「イカサマさいころ」か？

確率的 ARMS で反応規則を選ぶさいころの目の出方は、反応規則の適用回数の期待値によって決まる。

その期待値は基質の濃度によって変化していく。だから「イカサマ」なのだ。

濃度が高い分子ほど反応に使われやすく、さいころの目が変化していく。

反応速度論：どの反応が起きやすいか

反応速度論とは化学反応による濃度変化の動学であるが、直感的には「反応基質の濃度が高ければ反応しやすい（反応速度が速くなる）」し、「濃度が低ければ」反応がしにくくなる（反応速度が遅くなる）。

「なぜ、濃度が高くなると反応がしやすくなるのだろうか？」「濃度が高い」とは溶液中にその基質が多く溶け込んでいることを示している。よってそれだけ溶液中で「出会いやすくなる」。だが、それらの基質が「味噌」分子のように水に比べて重い分子はやがて反応槽の底にたまってしまおう。反応速度論を考える場合には、溶液はよく攪拌されているとし「味噌分子」のような重い分子が反応

槽の底にたまってしまう場合は考えない（だが、もちろんそのような場合を考えることは可能である。それに MAS のような局所的な相互作用で系が時間発展していく場合にはエージェントの空間配置も重要になるだろう）。

さて、溶液中で分子同士が衝突する場合、！☆！☆ブバーーーーーン！☆！☆と思いきり衝突する場合もあるだろうし、その逆にちょっとかする程度に衝突する場合も考えられる。ここでの「どの程度、思いきり衝突するのか？」は反応をかंगाえる場合に重要である。なぜなら、化学反応を生じさせるには「そのためのエネルギー」が必要だからである。

「そのためのエネルギー」とは化学反応の生じやすさを与えるものである。もし、このエネルギーが小さい場合には「ちょっと強めに衝突すれば」反応が生じてしまう。その一方で、このエネルギーが大きい場合にはちょっと強めに衝突したぐらいでは反応は生じず、かなり「思いきりぶつからないと」反応が生じない。この「反応を生じさせるためのエネルギー」は反応の速度を左右する（もちろんそれだけで定まるのではなく、基質の濃度も大きく影響するわけだが...）。

では以上を数理モデル化してみよう。まず以下では基質の濃度...いま私たちは離散系で考えているから「分子の数」とみなしてもよいだろう、例えば基質 A の濃度（または分子の数）は $[A]$ のように $[,]$ を用いてあらわす。

例として $A, B \rightarrow B, B$ という反応（速度定数 k_2 ）を考える。A と B が衝突する組み合わせの総数は $[A] \times [B]$ 通りあり、そのうち k_2 の割合で反応が起きるので：

```
# 反応速度 = k × [A] × [B]
#
# [A]=5, [B]=2, k=0.1 のとき
# 反応速度 = 0.1 × 5 × 2 = 1.0

# 同じ分子が複数回使われる場合（例：A, A → B, k=0.1）
# 反応速度 = k × [A]^2 = 0.1 × 5^2 = 2.5

# 一般則：反応速度 = k × [A]^(左辺での A の数) × [B]^(左辺での B の数) × ...
```

pow 関数：ベキ乗を計算する

反応速度の計算では pow 関数を使う。左辺に含まれない分子（使用回数=0）は 1 として計算に含

めない。

```
pow <- function(x, a) {  
  if (a == 0) 1 else x ^ a  
}  
  
pow(5, 0)    # → 1      (左辺でこの分子を使わない)  
pow(5, 1)    # → 5      (左辺でこの分子を 1 個使う)  
pow(5, 2)    # → 25     (左辺でこの分子を 2 個使う)
```

mapply で反応速度をベクトル計算する

反応規則の左辺がベクトルで表されているとき、mapply を使って全分子のべき乗を一気に計算できる。

```
ms <- c(5, 2, 3)      # A:5 個, B:2 個, C:3 個  
lhs <- c(1, 1, 0)     # 規則: A + B → ...  
  
# mapply で各分子について pow を適用する  
# → c(pow(5,1), pow(2,1), pow(3,0)) = c(5, 2, 1)  
mapply(pow, ms, lhs)  
  
# prod で全部掛け合わせる → 5 × 2 × 1 = 10  
prod(mapply(pow, ms, lhs))  
  
# 速度定数をかければ反応速度  
k <- 0.1  
k * prod(mapply(pow, ms, lhs))    # → 1.0
```

演習 4-1 pow と反応速度の計算

- ① pow 関数を定義し、pow(3,0), pow(3,1), pow(3,2) を確認しなさい。
- ② ms <- c(5, 2, 3), lhs <- c(1, 1, 0) として
prod(mapply(pow, ms, lhs)) の結果を確認しなさい。
- ③ lhs <- c(2, 1, 0) (左辺が A+A+B のとき) に変えて計算しなさい。
結果はいくつになるか? なぜそうなるか考えなさい。

- ④ `ms <- c(0, 2, 3)` のとき (A が 0 個)、反応速度はいくつになるか。
この意味を考えなさい。

各反応規則の確率を計算する：prob_ls 関数

複数の反応規則がある場合、各規則の反応速度の比率から確率を計算する。これが「イカサマさいころ」の目の設定になる。

```
# prob_ls 関数：全規則の確率をまとめて計算する
prob_ls <- function(ms, LHS, rconst) {
  n <- nrow(LHS) # 反応規則の数
  nume <- numeric(n) # 各規則の期待値を格納
  for (i in 1:n) {
    lhs <- LHS[i, ]
    nume[i] <- prod(mapply(pow, ms, lhs)) * rconst[i]
  }
  if (sum(nume) == 0) return(rep(0, n)) # 全速度が 0 のとき
  nume / sum(nume) # 確率に正規化して返す
}
```

演習 4-2 prob_ls 関数を確認する

- ① `prob_ls` 関数を定義しなさい (`pow` 関数も必要)。
- ② 以下の設定で `prob_ls` を実行して確率を確認しなさい。
- ```
ms <- c(5, 2)
LHS <- matrix(c(1,0, 1,1, 0,1), nrow=3, ncol=2, byrow=TRUE)
rconst <- c(0.1, 0.1, 0.1)
prob_ls(ms, LHS, rconst)
```
- ③ `ms <- c(0, 2)` (A が 0 個) のとき、確率はどう変化するか。
- ④ `rconst <- c(0.5, 0.1, 0.1)` に変えると確率はどう変化するか。

## ルーレット選択：イカサマさいころを振る

確率のリストをもとに、どの反応規則を選ぶかをランダムに決める。これが「さいころを振る」操作

になる。

```
btw 関数：乱数 rnum が累積確率リスト p_ls のどの区間に入るか返す
btw <- function(rnum, p_ls) {
 for (i in 1:length(p_ls)) {
 if (rnum <= p_ls[i]) return(i)
 }
 length(p_ls)
}

roulette 関数：確率リストから1つの規則番号をランダムに選ぶ
roulette <- function(prob) {
 p_ls <- cumsum(prob) # 累積確率
 rand <- runif(1) # 0~1 の乱数
 btw(rand, p_ls)
}

使用例
prob <- c(0.29, 0.59, 0.12)
roulette(prob) # → 1, 2, 3 のいずれかがランダムに戻る
```

#### 演習 4-3 ルーレット選択を確認する

- ① btw 関数と roulette 関数を定義しなさい。
- ② prob <- c(0.29, 0.59, 0.12) として roulette(prob) を 100 回実行しなさい。  
ヒント：table(replicate(100, roulette(prob)))
- ③ 確率の比率と実際の選択頻度が対応していることを確認しなさい。

## 確率的 ARMS のシミュレーター：全コード

ここまでの部品を組み合わせで完成させる。

```
---- pow 関数 ----
pow <- function(x, a) { if (a == 0) 1 else x ^ a }

---- 確率を計算する ----
prob_ls <- function(ms, LHS, rconst) {
 n <- nrow(LHS)
 nume <- numeric(n)
 for (i in 1:n) { nume[i] <- prod(mapply(pow, ms, LHS[i,])) * rconst[i] }
 if (sum(nume) == 0) return(rep(0, n))
 nume / sum(nume)
}
```

```

---- ルーレット選択 ----
btw <- function(rnum, p_ls) {
 for (i in 1:length(p_ls)) { if (rnum <= p_ls[i]) return(i) }
 length(p_ls)
}
roulet <- function(prob) { btw(runif(1), cumsum(prob)) }

---- 1 ステップの書き換え ----
arms_1step <- function(ms, LHS, RHS, rconst) {
 prob <- prob_ls(ms, LHS, rconst)
 if (sum(prob) == 0) return(ms)
 r <- roulette(prob)
 ms = LHS[r,] + RHS[r,]
}

---- 複数ステップ実行 ----
stochastic_arms <- function(ms, LHS, RHS, rconst, steps) {
 for (t in 1:steps) {
 ms <- arms_1step(ms, LHS, RHS, rconst)
 cat("step", t, ":", ms, "\n")
 }
 ms
}

```

## 実際に動かす : Lotka-Volterra モデル

以下の 3 つの反応規則は Lotka-Volterra モデル（被食者-捕食者系）の化学反応版である。A を被食者、B を捕食者と考えることができる。

```

規則 1: $A \rightarrow A, A$ (被食者が増える) k1 = 0.1
規則 2: $A, B \rightarrow B, B$ (捕食者が A を食べて増える) k2 = 0.1
規則 3: $B \rightarrow$ (捕食者が死ぬ) k3 = 0.1

set.seed(42)
ms <- c(10, 10)

LHS <- matrix(c(
 1, 0,
 1, 1,
 0, 1
), nrow=3, ncol=2, byrow=TRUE)

RHS <- matrix(c(
 2, 0,
 0, 2,
 0, 0
), nrow=3, ncol=2, byrow=TRUE)

```

```
rconst <- c(0.1, 0.1, 0.1)

result <- stochastic_arms(ms, LHS, RHS, rconst, 100)
print(result)
```

#### 演習 4-4 確率的 ARMS を動かす

上のコードをすべて入力して動かして下さい。

- ① `set.seed(42)` で 100 ステップ実行して結果を確認して下さい。
- ② `set.seed` を変えて再度実行して下さい。結果は変わりますか？なぜですか？
- ③ `rconst <- c(0.5, 0.1, 0.1)` に変えて実行して下さい。  
A と B の個体数はどのように変化しましたか？
- ④ `ms <- c(10, 0)` (捕食者がいない場合) から始めるとどうなりますか？

#### 演習 4-5 発展 : BZ 反応 (Belousov-Zhabotinskii 反応)

以下は化学振動系のモデルである (A, B, X, Y の 4 種類の分子)。

規則 1:  $A, X \rightarrow X, X$   $k_1$

規則 2:  $B, X \rightarrow Y$   $k_2$

規則 3:  $X, X, Y \rightarrow X, X, X$   $k_3$

規則 4:  $X \rightarrow$   $k_4$

確率的 ARMS を実装してシミュレーションして下さい。

$k_2, k_3$  の値を変えると振る舞いはどう変わるか調べ下さい。

(パラメータをうまく設定すると周期振動が現れる)